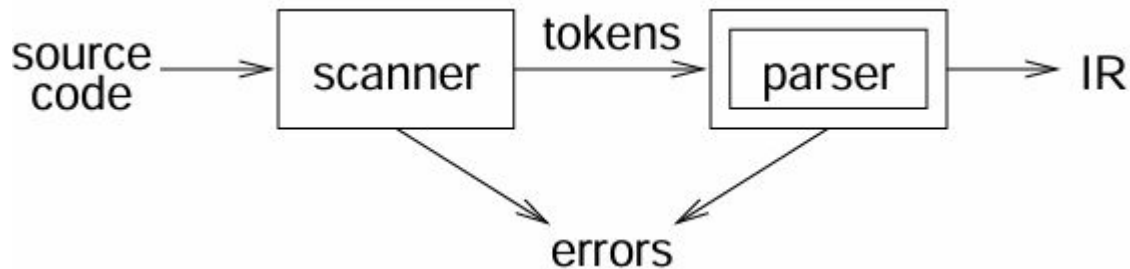


3장 파싱 (Parsing)

최광훈

전남대학교

프런트엔드(Front end)



파서

- 문맥 자유(context-free) 구문 분석

- * 토큰의 나열이 올바른 문법 구조를 이루는지

- 문맥에 민감한 분석을 가이드

- * 타입 검사, 선언 여부 확인 등 의미 분석을 안내

- IR (중간 표현)을 생성

- 오류 처리

- * 의미 있는 오류 메시지를 생성
- * 오류 복구를 시도

목차

LL 파싱

[실습] MiniJava 파서 (JavaCC)

LR 파싱

구문 분석(Syntax analysis)

문맥 자유 문법(Context-free grammar, CFG) (S, V_N, V_T, P)

- V_N : 비단말(Nonterminals) 기호 집합

- * 언어 안에 나타나는 (부분) 문자열들의 집합을 나타내는 구문 변수

- * 문법에 구조를 부여하는 역할

- V_T : 단말(Terminals) 기호 집합

- * 스캐너가 반환하는 토큰들의 집합

- ...

구문 분석(Syntax analysis)

문맥 자유 문법(Context-free grammar, CFG) (S, V_N, V_T, P)

- ...
- S : 시작 기호 (비단말)
- P : 생산 규칙(productions) 집합
 - * 단말과 비단말을 어떻게 결합하여 언어의 문자열을 만들 수 있는지
 - * 각 생성 규칙의 왼편(left-hand side)는 1개의 비단말만 올 수 있음 $A \rightarrow \dots$
- $V = V_N \cup V_T$: 문법 G 의 어휘

[참고] 표기법과 용어

$a, b, c \in V_t$ 단말 기호

$A, B, C \in V_n$ 비단말 기호

$U, V, W \in V$ 모든 기호

$\alpha, \beta, \gamma \in V^*$ 0이상 반복 기호 열

$u, v, w \in V_t^*$ 0이상 반복 단말 기호
열

- 단일 유도 (a single-step derivation)

: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \rightarrow \gamma$

- 다중 유도 ≥ 0 또는 ≥ 1

: $\Rightarrow^*$ 또는 $\Rightarrow^+$

- 문장 형태(sentential forms)

: $S \Rightarrow^* \beta$

- 문장(sentence)와 언어

: $L(G) = \{ w \in V_t^* \mid S \Rightarrow^+ w \}$

구문 분석

구문 분석을 위한 문법 작성 형식 : 백커스 나우어 폼 (Backus-Naur form, BNF)

예: $\langle \text{goal} \rangle ::= \langle \text{expr} \rangle$

$$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$$
$$| \text{ num}$$
$$| \text{ id}$$
$$\langle \text{op} \rangle ::= + \mid - \mid * \mid /$$

숫자와 변수를 사용하는 간단한 식의 언어를 기술하는 문법

비단말 ($\langle \dots \rangle$ 또는 대문자), 단말 기호 (typewriter 폰트, 밑줄), 생산규칙

스캐닝 vs 파싱

스캐닝과 파싱의 구분은 어디인가?

$$\begin{aligned} \text{term} &::= [\text{a} - \text{zA} - \text{z}]([\text{a} - \text{zA} - \text{z}] \mid [0 - 9])^* \\ &\quad \mid 0 \mid [1 - 9][0 - 9]^* \\ \text{op} &::= + \mid - \mid * \mid / \\ \text{expr} &::= (\text{term op})^* \text{term} \end{aligned}$$

- 정규식은 변수, 숫자, 키워드를 분류하는데 사용
- 정규식은 토큰을 기술하는데 (문법보다) 간단
- 정규식으로부터 (문법으로부터 보다) 빠른 스캐너를 만들 수 있음

스캐닝 vs 파싱

스캐닝과 파싱의 구분은 어디인가?

- 문맥 자유 문법은 괄호 (), **begin ... end**, **if ... then ... else** 매칭에 사용
- 문맥 자유 문법은 식의 구조를 부여하는 역할

구문 분석은 상당히 복잡함 (**C** 문법은 300여개의 생산 규칙으로 구성)

어휘 분석을 구문 분석에서 별도 단계로 분리하면 컴파일러를 유지 보수하는데
장점

유도 (Derivations)

CFG의 생산 규칙을 기호를 재작성하는 규칙(rewriting rules)으로 볼 수 있음

앞서 숫자와 변수 식에 대한 CFG 문법을 활용하여 $x + 2 * y$ 를 유도하는 과정

$\langle \text{goal} \rangle \Rightarrow \langle \text{expr} \rangle$

$\Rightarrow \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

...

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{num}, 2 \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{num}, 2 \rangle * \langle \text{expr} \rangle$

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

- 일련의 재작성 과정($\Rightarrow$)을 유도(a derivation) 또는 파스(a parse)라고 한다

- 그 유도 과정을 찾아내는 작업을 파싱(parsing)이라고 함.

$\langle \text{goal} \rangle \Rightarrow^* \text{id} + \text{num} * \text{id}$

유도 (Derivations)

$\langle \text{goal} \rangle \Rightarrow \langle \text{expr} \rangle$

$\Rightarrow \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

$\Rightarrow \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

$\Rightarrow \langle \text{id}, x \rangle \langle \text{op} \rangle \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{num}, 2 \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{num}, 2 \rangle * \langle \text{expr} \rangle$

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

유도(Derivations)

각 단계에서 재작성으로 대체할 비단말 기호를 선택

여러 비단말 기호가 있을 경우 이 선택에 따라 서로 다른 유도가 만들어질 수 있음

특히 중요한 두 가지 유도 방식

- 좌측 유도 (Leftmost derivation) : 매 단계마다 좌측의 비단말을 대체
- 우측 유도 (Rightmost derivation): 매 단계마다 우측의 비단말을 대체

앞의 예시는 좌측 유도였음!

유도 (Derivations)

좌측 유도 (Leftmost derivations)

<goal> => <expr>

=> <expr> <op> <expr>

=> <expr> <op> <expr> <op> <expr>

=> <id,x> <op> <expr> <op> <expr>

=> <id,x> + <expr> <op> <expr>

=> <id,x> + <num,2> <op> <expr>

=> <id,x> + <num,2> * <expr>

=> <id,x> + <num,2> * <id,y>

<goal> =>* id + num * id

유도 (Derivations)

우측 유도 (Rightmost derivations)

$\langle \text{goal} \rangle \Rightarrow \langle \text{expr} \rangle$

$\Rightarrow \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

$\Rightarrow \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{expr} \rangle * \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle * \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{expr} \rangle + \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

$\langle \text{goal} \rangle \Rightarrow^* \text{id} + \text{num} * \text{id}$

우선순위 (Precedence)

오른쪽과 같이 $x + 2 * y$ 의 파스 트리를 만들고
트리를 따라 계산해보면(Treewalk evaluation)

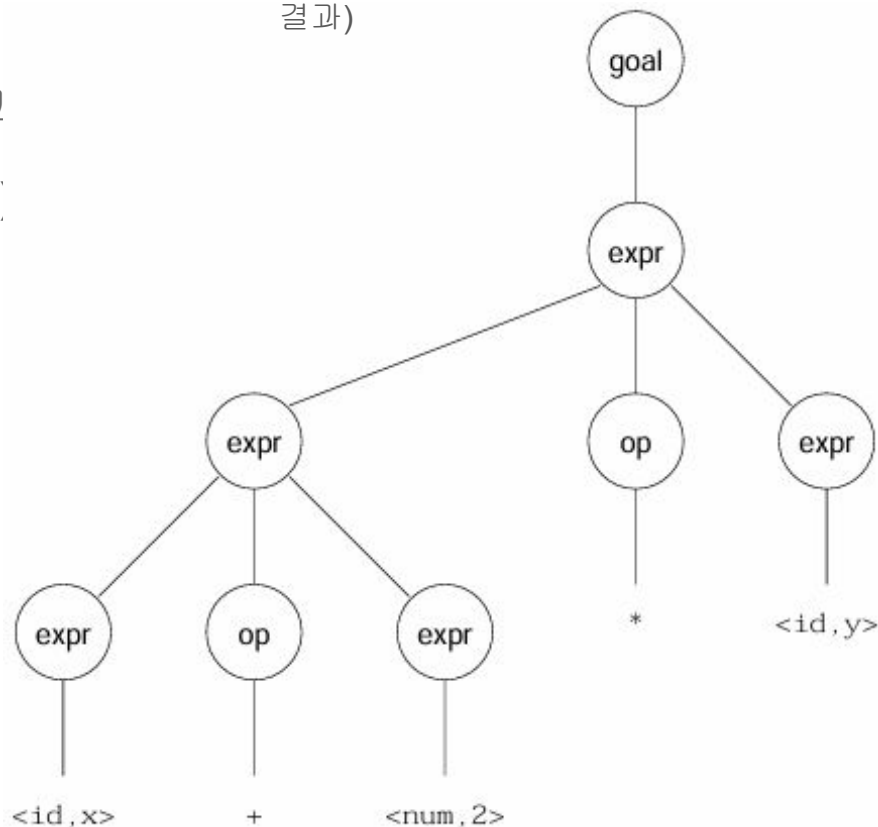
$$(x + 2) * y$$

=> 계산 순서가 잘못됨!

=> $x + (2 * y)$ 를 계산해야함

Q. $x + (2 * y)$ 를 계산하는 파스 트리?

(좌측 유도
결과)



우선순위 (Precedence)

앞의 예제 문법은 $x + 2 * y$ 에 대해 두 가지 유도, $(x+2)*y$ 파스 트리와 $x+(2*y)$ 파스 트리가 가능

우선 순위(괄호가 없을 때 암묵적인 계산 순서)에 대한 개념이 없음

우선 순위를 고려해서 다시 작성한 문법:

```
1 | <goal> ::= <expr>
2 | <expr> ::= <expr> + <term>
3 |           | <expr> - <term>
4 |           | <term>
5 | <term>  ::= <term> * <factor>
6 |           | <term> / <factor>
7 |           | <factor>
8 | <factor> ::= num
9 |           | id
```

이 문법은 유도 과정에 우선순위를 강제한다.

- <term>은 <expr>에서 유도되고,
- <factor>는 <term>에서 유도된다!

우선순위 (Precedence)

새로운 문법으로 $x + 2 * y$ 를 유도하면:

$\langle \text{goal} \rangle \Rightarrow \langle \text{expr} \rangle$

$\Rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$

$\Rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle * \langle \text{factor} \rangle$

$\Rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle * \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{expr} \rangle + \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{term} \rangle + \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{factor} \rangle + \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

$\Rightarrow \langle \text{id}, x \rangle + \langle \text{num}, 2 \rangle * \langle \text{id}, y \rangle$

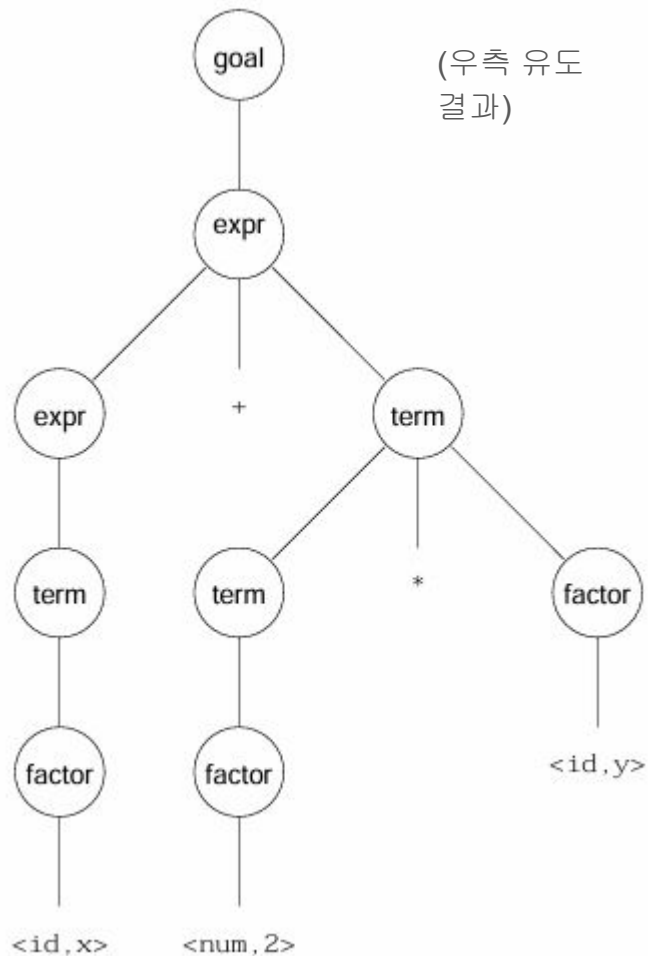
$\langle \text{goal} \rangle \Rightarrow * \text{id} + \text{num} * \text{id}$

이번에는 $x + (2 * y)$ 파스 트리를
만들

우선순위 (Precedence)

오른쪽 파스 트리를 따라 계산하면

$$x + (2 * y)$$



우선순위 (Precedence)

Q. $x + 2 + y$

- P.16의 문법의 좌측 유도(또는, 우측 유도에서) 좌결합 $(x+2)+y$ 파스 트리를 확인하시오.

Q. $z = w = 2$

- 할당식을 허용하고, 할당식은 우결합 $z = (w = 2)$ 이 되도록 P.16 문법을 수정하시오.

- $z = w = 2$ 를 유도해보고 파스 트리를 확인하시오.

모호함(Ambiguity)

어떤 문법이 하나의 동일한 문장 형태(sentential form)에 대해 둘 이상의 유도 과정을 허용한다면 그 문법은 모호하다(ambiguous).

예) $\langle \text{stmt} \rangle ::= \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{stmt} \rangle$

| $\text{if } \langle \text{expr} \rangle \text{ then } \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle$

| other stmts

$\text{if } E_1 \text{ then if } E_2 \text{ then } S_1 \text{ else } S_2$ 문장을 유도하는 과정이 두 가지 있음

이 모호함은 의미 해석 이전에 발생하는 구문 문법에서 비롯된 것이고,
문맥 자유 문법에서의 모호함이다.

모호함(Ambiguity)

문법을 재구성해서 모호함을 없앨 수도 있음

예) $\langle \text{stmt} \rangle ::= \langle \text{matched} \rangle \mid \langle \text{unmatched} \rangle$

$\langle \text{matched} \rangle ::= \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{matched} \rangle \text{ else } \langle \text{matched} \rangle \mid \text{other stmts}$

$\langle \text{unmatched} \rangle ::= \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{stmt} \rangle$

$\mid \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{matched} \rangle \text{ else } \langle \text{unmatched} \rangle$

- 새로운 문법은 앞서 모호한 문법과 동일한 언어를 생성
 - **else**를 가장 가까운 미결합 **then**에 연결하도록 구성함
- => 일반적으로 받아들여지는 해석

모호함(Ambiguity)

모호함은 보통 문맥 자유 문법 명세에서의 혼동 때문에 발생

문맥 의존에서도 혼동이 발생할 수 있음: 예) 오버로딩(overloading)

예) `a = f ( 17 ); b = f (17, 'a');`

(f는 int 인자의 함수 또는 int 인자와 char 인자 두 개를 받는 함수)

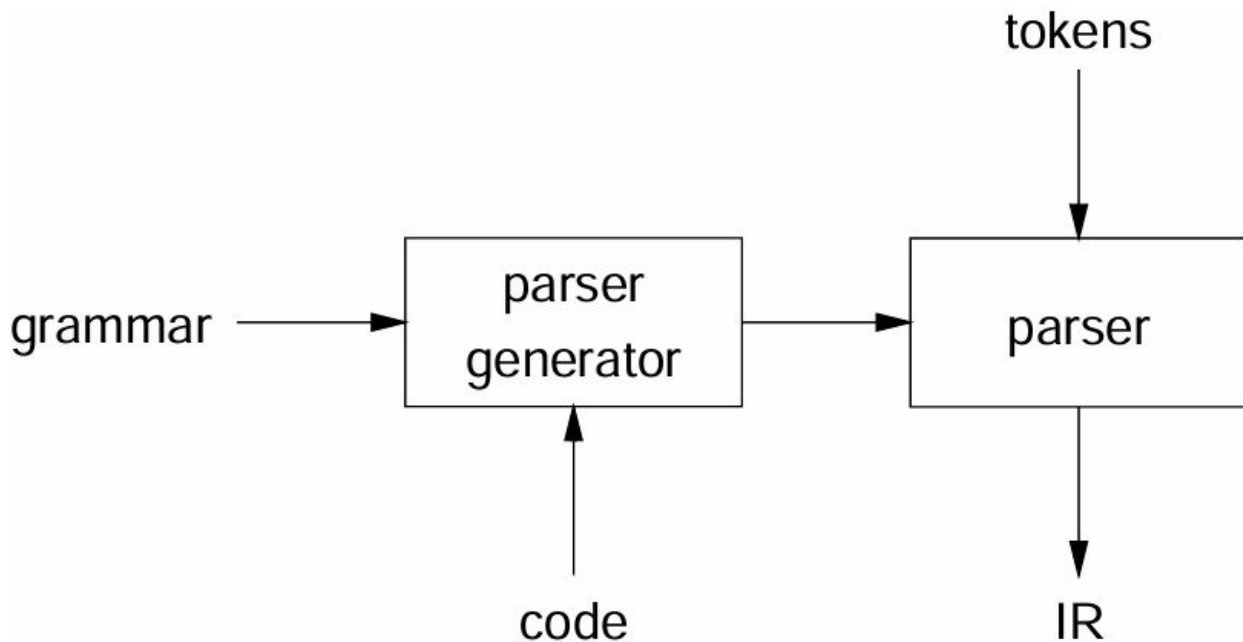
‘f’를 구분하려면 함수 f의 선언과 같은 문맥이 필요

- 문맥 자유의 영역이 아님
- 함수 f의 타입의 정보를 이용하는 문제

=> 파싱을 더 복잡하게 만들기보다는, 이 문제는 별도의 단계에서 처리하는 것이 보통.

파싱 (Parsing): 전체 흐름

우리의 목표는 파서를 만들어내는 유연한 시스템을 만들기!



하향(Top-down) vs. 상향(Bottom-up)

하향 파서(Top-down parsers)

- 유도 트리의 루트에서 시작하여 아래로 채워 나감
- 생성 규칙을 선택하고 입력과의 일치를 시도함
- 경우에 따라 백트래킹이 필요
- 일부 문법은 백트래킹 없이 처리 가능함(예측 파싱 predictive parsing)

상향식 파서 (Bottom-up parsers)

- 유도 트리의 밑에서 시작하여 위로 채워 나감
- 올바른 첫 토큰에 대해 유효한 상태에서 시작함
- 입력을 소비하면서 가능한 경우를 인코딩하도록 상태를 변경함 (**valid prefixes**를 인식함)
- 상태와 문장 형태(**sentential forms**)를 저장하기 위해 스택을 사용함

하향 파싱(Top-down parsing)

- 하향 파서는 구문 분석 트리의 루트, 즉 문법의 시작 기호에서 시작함
- 구문 분석 트리의 **leaf sequence**가 입력 문자열과 일치할 때까지 아래 단계를 반복
 1. **A**의 노드에서, 생성 규칙 **A $\rightarrow$ α** 를 선택, **α** 의 각 기호에 알맞은 자식 노드를 생성함
 2. **leaf sequence**에 추가된 단말 기호가 입력 문자열과 일치하지 않으면 백트래킹
 3. 다음 확장할 노드(비단말 기호)를 찾음
- 핵심은 1단계에서 **입력 문자열의 안내에 따라** 올바른 생성 규칙을 선택하는 것

간단 식의 문법

입력 문자열: $x - 2 * y$

1	$\langle \text{goal} \rangle$	$::=$	$\langle \text{expr} \rangle$
2	$\langle \text{expr} \rangle$	$::=$	$\langle \text{expr} \rangle + \langle \text{term} \rangle$
3		$ $	$\langle \text{expr} \rangle - \langle \text{term} \rangle$
4		$ $	$\langle \text{term} \rangle$
5	$\langle \text{term} \rangle$	$::=$	$\langle \text{term} \rangle * \langle \text{factor} \rangle$
6		$ $	$\langle \text{term} \rangle / \langle \text{factor} \rangle$
7		$ $	$\langle \text{factor} \rangle$
8	$\langle \text{factor} \rangle$	$::=$	num
9		$ $	id

예제

핵심은 1단계에서

입력 문자열의

안내에 따라

올바른 생성 규칙을

선택하는 것!

Prod'n	Sentential form	Input
-	$\langle \text{goal} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
1	$\langle \text{expr} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
2	$\langle \text{expr} \rangle + \langle \text{term} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
4	$\langle \text{term} \rangle + \langle \text{term} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
7	$\langle \text{factor} \rangle + \langle \text{term} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
9	$\text{id} + \langle \text{term} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
-	$\text{id} + \langle \text{term} \rangle$	$x \quad \uparrow - \quad 2 \quad * \quad y$
-	$\langle \text{expr} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
3	$\langle \text{expr} \rangle - \langle \text{term} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
4	$\langle \text{term} \rangle - \langle \text{term} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
7	$\langle \text{factor} \rangle - \langle \text{term} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
9	$\text{id} - \langle \text{term} \rangle$	$\uparrow x \quad - \quad 2 \quad * \quad y$
-	$\text{id} - \langle \text{term} \rangle$	$x \quad \uparrow - \quad 2 \quad * \quad y$
-	$\text{id} - \langle \text{term} \rangle$	$x \quad - \quad \uparrow 2 \quad * \quad y$
7	$\text{id} - \langle \text{factor} \rangle$	$x \quad - \quad \uparrow 2 \quad * \quad y$
8	$\text{id} - \text{num}$	$x \quad - \quad \uparrow 2 \quad * \quad y$
-	$\text{id} - \text{num}$	$x \quad - \quad 2 \quad \uparrow * \quad y$
-	$\text{id} - \langle \text{term} \rangle$	$x \quad - \quad \uparrow 2 \quad * \quad y$
5	$\text{id} - \langle \text{term} \rangle * \langle \text{factor} \rangle$	$x \quad - \quad \uparrow 2 \quad * \quad y$
7	$\text{id} - \langle \text{factor} \rangle * \langle \text{factor} \rangle$	$x \quad - \quad \uparrow 2 \quad * \quad y$
8	$\text{id} - \text{num} * \langle \text{factor} \rangle$	$x \quad - \quad \uparrow 2 \quad * \quad y$
-	$\text{id} - \text{num} * \langle \text{factor} \rangle$	$x \quad - \quad 2 \quad \uparrow * \quad y$
-	$\text{id} - \text{num} * \langle \text{factor} \rangle$	$x \quad - \quad 2 \quad * \quad \uparrow y$
9	$\text{id} - \text{num} * \text{id}$	$x \quad - \quad 2 \quad * \quad \uparrow y$
-	$\text{id} - \text{num} * \text{id}$	$x \quad - \quad 2 \quad * \quad y \quad \uparrow$

예제

동일한 입력 $x - 2 * y$ 에 다른 유도

Prod'n	Sentential form	Input
–	$\langle \text{goal} \rangle$	$\uparrow x - 2 * y$
1	$\langle \text{expr} \rangle$	$\uparrow x - 2 * y$
2	$\langle \text{expr} \rangle + \langle \text{term} \rangle$	$\uparrow x - 2 * y$
2	$\langle \text{expr} \rangle + \langle \text{term} \rangle + \langle \text{term} \rangle$	$\uparrow x - 2 * y$
2	$\langle \text{expr} \rangle + \langle \text{term} \rangle + \dots$	$\uparrow x - 2 * y$
2	$\langle \text{expr} \rangle + \langle \text{term} \rangle + \dots$	$\uparrow x - 2 * y$
2	$\dots$	$\uparrow x - 2 * y$

잘못된 선택을 하면, 무한 반복 파싱 진행 (왜? 맨 왼쪽 $\langle \text{expr} \rangle$ 만 반복 확장하기 때문)

(But 파서는 반드시 종료되어야 함!)

왼쪽 재귀 구조(Left-recursion)

Top-down 파서는 문법에서 왼쪽 재귀 구조(left recursion) 를 처리할 수 없음.

(정의) 문법이 왼쪽 재귀 형태(left-recursive)

- $\exists A \in V_n$ such that $A \Rightarrow^+ A\alpha$ for some string α
- 비단말 기호 A 가 $A\alpha$ 로 유도되는, 그러한 A 가 존재하는 문법은 왼쪽 재귀 형태

앞서 살펴본 단순한 식 문법은 왼쪽 재귀 구조임!

왼쪽 재귀 구조를 제거하는 변환

왼쪽 재귀 구조를 제거하기 위해, 문법을 변환 (변환 전후 문법의 언어는 동일!)

왼쪽 재귀 구조의 문법 예시

G1: $\langle \text{foo} \rangle ::= \langle \text{foo} \rangle \alpha \mid \beta$ (전제: α 와 β 는 $\langle \text{foo} \rangle$ 로 시작하지 않음)

이 문법을 다음과 같이 다시 쓸 수 있음

G2: $\langle \text{foo} \rangle ::= \beta \langle \text{bar} \rangle$

$\langle \text{bar} \rangle ::= \alpha \langle \text{bar} \rangle \mid \epsilon$ ($\langle \text{bar} \rangle$ 는 새로 도입한 비단말 기호)

변환 후 문법은 왼쪽 재귀 구조가 포함되어 있지 않다!

Q. $L(G1) = L(G2)$?

왼쪽 재귀 구조를 제거하는 변환

왼쪽 재귀 구조가 있는 문법 G1

$$\begin{aligned}\langle \text{expr} \rangle &::= \langle \text{expr} \rangle + \langle \text{term} \rangle \\ &\quad | \langle \text{expr} \rangle - \langle \text{term} \rangle \\ &\quad | \langle \text{term} \rangle \\ \langle \text{term} \rangle &::= \langle \text{term} \rangle * \langle \text{factor} \rangle \\ &\quad | \langle \text{term} \rangle / \langle \text{factor} \rangle \\ &\quad | \langle \text{factor} \rangle\end{aligned}$$

그 구조를 제거하는 변환 후 문법 G2

$$\begin{aligned}\langle \text{expr} \rangle &::= \langle \text{term} \rangle \langle \text{expr}' \rangle \\ \langle \text{expr}' \rangle &::= + \langle \text{term} \rangle \langle \text{expr}' \rangle \\ &\quad | \epsilon \\ &\quad | - \langle \text{term} \rangle \langle \text{expr}' \rangle \\ \langle \text{term} \rangle &::= \langle \text{factor} \rangle \langle \text{term}' \rangle \\ \langle \text{term}' \rangle &::= * \langle \text{factor} \rangle \langle \text{term}' \rangle \\ &\quad | \epsilon \\ &\quad | / \langle \text{factor} \rangle \langle \text{term}' \rangle\end{aligned}$$

하향 파서가 G2를 따르면 구문 분석은 반드시 종료

종료 문제는 해결되었으나, 어떤 입력을 보고 backtrack하는 상황은 여전히 발생할 수 있음

동일 언어 vs 동일 의미

동일한 언어이지만 $L(G1) = L(G2) = L(G3)$, $G1/G2$ 의 파스 트리 $\neq G3$ 의 파스 트리

- $G3$: 연산자 오른쪽 결합(right associativity), $G1/G2$: 왼쪽 결합(left assoc.)

```
1  <goal>   ::= <expr>
2  <expr>   ::= <term> + <expr>
3           | <term> - <expr>
4           | <term>
5  <term>   ::= <factor> * <term>
6           | <factor> / <term>
7           | <factor>
8  <factor> ::= num
9           | id
```

	G1	G2	G3
언어	동일	동일	동일
하향식 파싱	불가 (무한 loop)	가능	가능
파스 트리	$G1=G2$ (왼 쪽결합)	$G1=G2$ (왼 쪽결합)	$\neq$ (오른쪽 결합)

최종 문법 G2

왼쪽 재귀 구조를 제거한 문법

1		$\langle \text{goal} \rangle$	$::=$	$\langle \text{expr} \rangle$
2		$\langle \text{expr} \rangle$	$::=$	$\langle \text{term} \rangle \langle \text{expr}' \rangle$
3		$\langle \text{expr}' \rangle$	$::=$	$+ \langle \text{term} \rangle \langle \text{expr}' \rangle$
4				$- \langle \text{term} \rangle \langle \text{expr}' \rangle$
5				ϵ
6		$\langle \text{term} \rangle$	$::=$	$\langle \text{factor} \rangle \langle \text{term}' \rangle$
7		$\langle \text{term}' \rangle$	$::=$	$* \langle \text{factor} \rangle \langle \text{term}' \rangle$
8				$/ \langle \text{factor} \rangle \langle \text{term}' \rangle$
9				ϵ
10		$\langle \text{factor} \rangle$	$::=$	num
11				id

얼마나 많은 lookahead가 필요한가?

하향 파서는 잘못된 생산 규칙을 선택하면 **backtrack** 할 수 있음

임의의 CFG를 파싱하려면 임의로 긴 lookahead가 필요한가?

- 일반적으로 Yes: Earley 알고리즘, Cocke-Younger-Kasami(CYK) 알고리즘

제한된 길이의 lookahead로 파싱할 수 있는 CFG도 상당히 많음

대부분의 프로그래밍 언어 구문이 이러한 CFG에 해당.

대표적으로,

- LL(1) : **L**eft-to-right scan, **L**eft-most derivation, **1**-token lookahead
- LR(1) : **L**eft-to-right scan, **R**ight-most derivation, **1**-token lookahead

예측 파싱(Predictive parsing)

기본 아이디어

- 비단말 A 에 대해 두 개의 생산 규칙 $A \rightarrow \alpha$, $A \rightarrow \beta$ 가 있다면,
- 파서가 올바른 생산 규칙을 구별해서 선택하는 방법이 필요

$FIRST(\alpha)$: α 에서 유도되는 어떤 문자열에서 처음 나타나는 단말기호들(의 집합)

- $\alpha \Rightarrow^* a \vee^*$ 이면, $a \in FIRST(\alpha)$ (단말기호 $a \in V_t$)

예측 파싱(Predictive parsing)

...

$\text{FIRST}(\alpha)$: α 에서 유도되는 어떤 문자열에서 처음 나타나는 단말기호들(의 집합)

- $\alpha \Rightarrow^* a \vee^*$ 이면, $a \in \text{FIRST}(\alpha)$ (단말기호 $a \in V_t$)

예측 파싱의 **Key** 조건:

- 두 생산 규칙이 함께 존재할 때, $A \rightarrow \alpha, A \rightarrow \beta$,

- 예측 파싱을 위해 필요한 조건: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$

$\Rightarrow$ 이 조건으로, 파서는 오직 1개의 기호로 된 lookahead로 올바른 선택을 할 수 있음

간단식의 문법 **G2**는 이 조건을 만족!

Left factoring (좌인수 분해)

주어진 문법이 앞의 **FIRST**에 대한 **Key** 성질을 만족하지 않는 경우, 좌인수 분해로 **Key** 성질을 만족하는 문법으로 변환할 수도 있음 (항상 가능한 것은 아님)

좌인수 분해

- 비단말 **A**의 여러 선택지들이 공통 **prefix** α 를 가지면, 그 공통 부분을 묶어냄

$$A \rightarrow \alpha \beta_1 \mid \dots \mid \alpha \beta_n \Rightarrow A \rightarrow \alpha A' \text{ 과 } A' \rightarrow \beta_1 \mid \dots \mid \beta_n \text{ (A'은 새로운 비단말 기호)}$$

- 하나의 비단말에 대해 두 개 이상의 선택지가 공통 **prefix**를 더 이상 가지지 않을 때까지 이 과정을 반복 반복

Left factoring (좌인수 분해)

간단 식 문법 G3 (우측 재귀 구조)

1		$\langle \text{goal} \rangle$	$::=$	$\langle \text{expr} \rangle$
2		$\langle \text{expr} \rangle$	$::=$	$\langle \text{term} \rangle + \langle \text{expr} \rangle$
3				$\langle \text{term} \rangle - \langle \text{expr} \rangle$
4				$\langle \text{term} \rangle$
5		$\langle \text{term} \rangle$	$::=$	$\langle \text{factor} \rangle * \langle \text{term} \rangle$
6				$\langle \text{factor} \rangle / \langle \text{term} \rangle$
7				$\langle \text{factor} \rangle$
8		$\langle \text{factor} \rangle$	$::=$	num
9				id

생산 규칙 2, 3, 4를 구별하려면, num이나 id를 넘어서 +, -, *, /를 확인해야 함

$$\text{FIRST}(2) \cap \text{FIRST}(3) \cap \text{FIRST}(4) \neq \emptyset$$

이 문법은 **Key** 성질을 갖고 있지 않음!

Left factoring (좌인수 분해)

두 단말 기호 $\langle \text{expr} \rangle$ 과 $\langle \text{term} \rangle$

$$\begin{aligned}\langle \text{expr} \rangle &::= \langle \text{term} \rangle + \langle \text{expr} \rangle \\ &\quad | \langle \text{term} \rangle - \langle \text{expr} \rangle \\ &\quad | \langle \text{term} \rangle \\ \langle \text{term} \rangle &::= \langle \text{factor} \rangle * \langle \text{term} \rangle \\ &\quad | \langle \text{factor} \rangle / \langle \text{term} \rangle \\ &\quad | \langle \text{factor} \rangle\end{aligned}$$

좌인수 분해를 적용한 다음,

$$\begin{aligned}\langle \text{expr} \rangle &::= \langle \text{term} \rangle \langle \text{expr}' \rangle \\ \langle \text{expr}' \rangle &::= + \langle \text{expr} \rangle \\ &\quad | - \langle \text{expr} \rangle \\ &\quad | \epsilon \\ \langle \text{term} \rangle &::= \langle \text{factor} \rangle \langle \text{term}' \rangle \\ \langle \text{term}' \rangle &::= * \langle \text{term} \rangle \\ &\quad | / \langle \text{term} \rangle \\ &\quad | \epsilon\end{aligned}$$

Left factoring (좌인수 분해)

좌인수 분해 결과, G3는 아래의 문법 G4로 변환됨(우측 재귀 구조는 그대로)

1		$\langle \text{goal} \rangle$	::=	$\langle \text{expr} \rangle$
2		$\langle \text{expr} \rangle$	::=	$\langle \text{term} \rangle \langle \text{expr}' \rangle$
3		$\langle \text{expr}' \rangle$	::=	$+\langle \text{expr} \rangle$
4				$-\langle \text{expr} \rangle$
5				ϵ
6		$\langle \text{term} \rangle$	::=	$\langle \text{factor} \rangle \langle \text{term}' \rangle$
7		$\langle \text{term}' \rangle$	::=	$*\langle \text{term} \rangle$
8				$/\langle \text{term} \rangle$
9				ϵ
10		$\langle \text{factor} \rangle$	::=	num
11				id

단 하나의 토큰 lookahead만으로도 생산 규칙을 선택할 수 있음!

Left factoring (좌인수 분해)

다음 기호만 보면

각 선택을 올바르게

결정할 수 있음

	Sentential form	Input
-	$\langle \text{goal} \rangle$	$\uparrow x - 2 * y$
1	$\langle \text{expr} \rangle$	$\uparrow x - 2 * y$
2	$\langle \text{term} \rangle \langle \text{expr}' \rangle$	$\uparrow x - 2 * y$
6	$\langle \text{factor} \rangle \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$\uparrow x - 2 * y$
11	$\text{id} \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$\uparrow x - 2 * y$
-	$\text{id} \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$x \uparrow - 2 * y$
9	$\text{id} \epsilon \langle \text{expr}' \rangle$	$x \uparrow - 2$
4	$\text{id} - \langle \text{expr} \rangle$	$x \uparrow - 2 * y$
-	$\text{id} - \langle \text{expr} \rangle$	$x - \uparrow 2 * y$
2	$\text{id} - \langle \text{term} \rangle \langle \text{expr}' \rangle$	$x - \uparrow 2 * y$
6	$\text{id} - \langle \text{factor} \rangle \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$x - \uparrow 2 * y$
10	$\text{id} - \text{num} \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$x - \uparrow 2 * y$
-	$\text{id} - \text{num} \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$x - 2 \uparrow * y$
7	$\text{id} - \text{num} * \langle \text{term} \rangle \langle \text{expr}' \rangle$	$x - 2 \uparrow * y$
-	$\text{id} - \text{num} * \langle \text{term} \rangle \langle \text{expr}' \rangle$	$x - 2 * \uparrow y$
6	$\text{id} - \text{num} * \langle \text{factor} \rangle \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$x - 2 * \uparrow y$
11	$\text{id} - \text{num} * \text{id} \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$x - 2 * \uparrow y$
-	$\text{id} - \text{num} * \text{id} \langle \text{term}' \rangle \langle \text{expr}' \rangle$	$x - 2 * y \uparrow$
9	$\text{id} - \text{num} * \text{id} \langle \text{expr}' \rangle$	$x - 2 * y \uparrow$
5	$\text{id} - \text{num} * \text{id}$	$x - 2 * y \uparrow$

다시 왼쪽 재귀 구조 제거

전형적인 단계: 좌인수 분해 변환 $\rightarrow$ 왼쪽 재귀 구조를 제거 $\rightarrow$ LL(1)

- 비단말 A의 생산규칙에 왼쪽 재귀 구조가 있다면,

$$A \rightarrow A\alpha \mid \beta \mid \dots \mid \gamma$$

$$\Rightarrow A \rightarrow NA'$$

$$N \rightarrow \beta \mid \dots \mid \gamma$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

- 왼쪽 재귀 구조가 더 이상 남아 있지 않을 때까지 이 과정을 반복

일반성

Q. 좌인수분해와 왼쪽 재귀 구조를 제거하면 모든 문맥자유문법(CFG)을 하나의 토큰 **lookahead**만으로 예측 파싱할 수 있는 형태로 변환할 수 있을까?

A. 주어진 문맥자유문법이 예측 파싱의 **Key** 조건을 만족하지 않을 때, 그와 동등한 (equivalent) 문법 중에서 그 조건을 만족하는 문법이 존재하는지 여부는 **undecidable** 하다.

예) $\{a^n 0 b^n \mid n \geq 1\} \cup \{a^n 1 b^{2n} \mid n \geq 1\}$

=> 어떤 유도를 선택할지 결정하려면, 0 또는 1이 나타날 때까지 임의의 개수의 **a**를 더 살펴봐야 함!

재귀 하강 파싱(Recursive descent parsing)

우결합 문법 G4로부터 구성한 재귀 하강 파서

```
goal:
  token ← next_token();
  if (expr() = ERROR | token ≠ EOF) then
    return ERROR;

expr:
  if (term() = ERROR) then
    return ERROR;
  else return expr_prime();

expr_prime:
  if (token = PLUS) then
    token ← next_token();
    return expr();
  else if (token = MINUS) then
    token ← next_token();
    return expr();
  else return OK;
```

```
term:
  if (factor() = ERROR) then
    return ERROR;
  else return term_prime();

term_prime:
  if (token = MULT) then
    token ← next_token();
    return term();
  else if (token = DIV) then
    token ← next_token();
    return term();
  else return OK;
```

재귀 하강 파싱(Recursive descent parsing)

우결합 문법 G4로부터 구성한 재귀 하강 파서

```
factor:
    if (token = NUM) then
        token ← next_token();
        return OK;
    else if (token = ID) then
        token ← next_token();
        return OK;
    else return ERROR;
```

파스 트리 만들기

파서의 중요한 역할은 소스 코드의 중간 표현 IR(AST)을 만드는 것

아래의 코드를 적절히 추가하여 추상 구문 트리(AST)를 만들 수 있음:

- factor()는 id와 num을 스택에 push하고
- term_prim()은 *, /을 스택에 push하고
- term()은 스택에서 3개를 pop하고 subtree를 만들어 다시 스택에 push
- expr_prime()은 +, -를 스택에 push하고
- expr()은 스택에서 3개를 pop하고 subtree를 만들어 다시 스택에 push
- goal()은 스택에서 pop한 트리를 리턴

비재귀 예측 파싱(Non-recursive predictive parsing)

관찰 Observation:

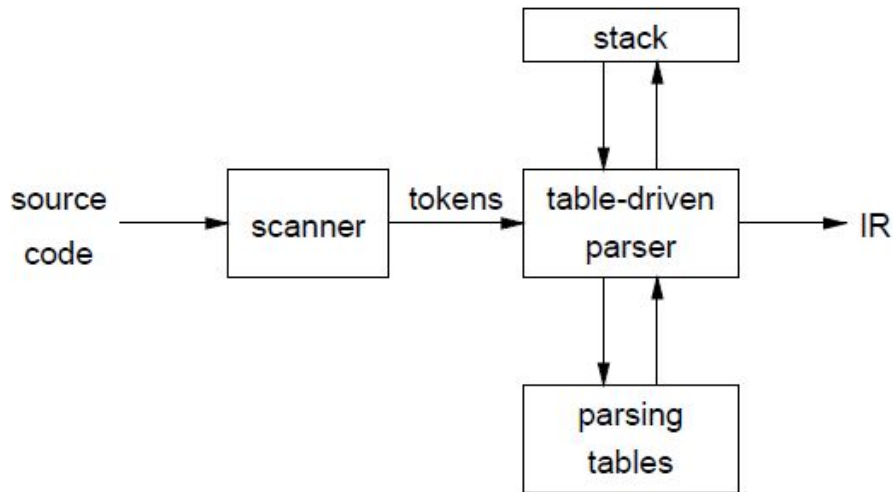
- 재귀 하강 파서는 상태 정보를, 파서를 실행할 때 스택(runtime stack), 즉 호출 스택(call stack)에 인코딩하고 있음.
- 하지만 스택 추상화를 구현하기 위해 재귀 함수 호출을 사용하는 방식은 그다지 효율적이지 않을 수 있음.

다른 구현 방법:

- 명시적 스택(explicit stack)을 사용하는 손으로 직접 작성한 파서
- 스택 기반(stack-based), 테이블 구동(table-driven) 파서

비재귀 예측 파싱(Non-recursive predictive parsing)

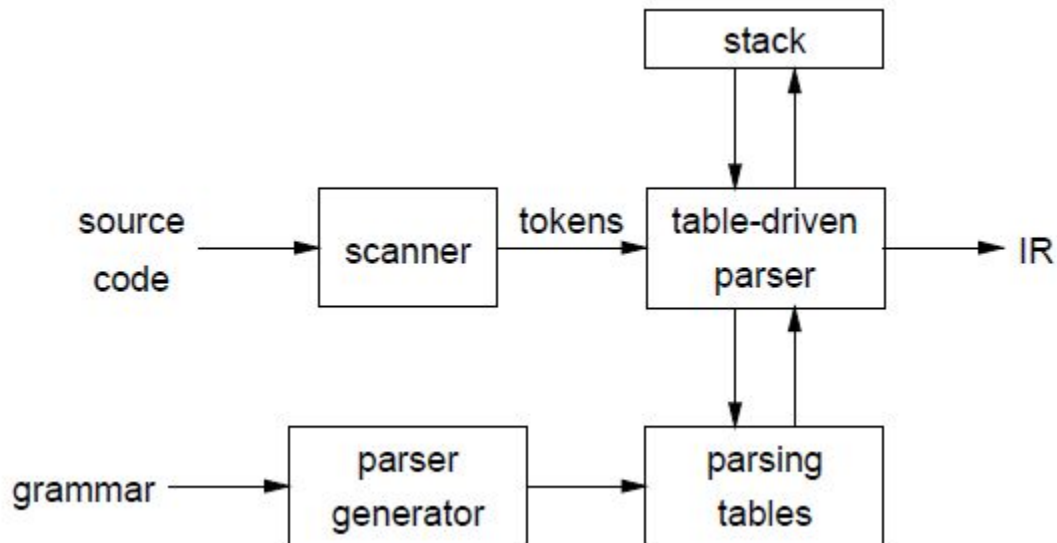
예측 파서의 구조



- 파서 코드를 작성하는 대신 테이블을 만듦
- 테이블 만들기를 자동화!

테이블을 사용하는 파서(Table-driven parsers)

파서 생성 시스템의 구조



이는 top-down (LL) 파서와 bottom-up (LR) 파서 모두에 해당한다.

비재귀 예측 파싱(Non-recursive predictive parsing)

입력:

- 문자열 w 와

- 파싱 테이블 M

(문법 G 의 M)

```
tos  $\leftarrow$  0
Stack[tos]  $\leftarrow$  EOF
Stack[++tos]  $\leftarrow$  Start Symbol
token  $\leftarrow$  next_token()
repeat
  X  $\leftarrow$  Stack[tos]
  if X is a terminal or EOF then
    if X = token then
      pop X
      token  $\leftarrow$  next_token()
    else error()
  else /* X is a non-terminal */
    if  $M[X, \text{token}] = X \rightarrow Y_1 Y_2 \cdots Y_k$  then
      pop X
      push  $Y_k, Y_{k-1}, \dots, Y_1$ 
    else error()
until X = EOF
```

비재귀 예측 파싱(Non-recursive predictive parsing)

지금 필요한 것은 파싱 테이블 M !

간단 식의 문법:

```
1  <goal> ::= <expr>
2  <expr>  ::= <term> <expr'>
3  <expr'> ::= + <expr>
4           | - <expr>
5           | ε
6  <term>   ::= <factor> <term'>
7  <term'>  ::= * <term>
8           | / <term>
9           | ε
10 <factor> ::= num
11          | id
```

파싱 테이블:

	id	num	+	-	*	/	$\†
<goal>	1	1	-	-	-	-	-
<expr>	2	2	-	-	-	-	-
<expr'>	-	-	3	4	-	-	5
<term>	6	6	-	-	-	-	-
<term'>	-	-	9	9	7	8	9
<factor>	11	10	-	-	-	-	-

FIRST

$\text{FIRST}(\alpha)$: α 에서 유도되는 어떤 문자열에서 처음 나타나는 단말기호들(의 집합)

- $\{ a \in V_t \mid \alpha \Rightarrow^* a \beta^* \}$

- 만일 $\alpha \Rightarrow^* \epsilon$ 이면 $\epsilon \in \text{FIRST}(\alpha)$

FIRST

기호 X 에 대하여, $\text{FIRST}(X)$ 를 구하는 알고리즘

1. If $X \in V_t$ then $\text{FIRST}(X)$ is $\{X\}$
 2. If $X \rightarrow \epsilon$ then add ϵ to $\text{FIRST}(X)$.
 3. If $X \rightarrow Y_1 Y_2 \cdots Y_k$:
 - (a) Put $\text{FIRST}(Y_1) - \{\epsilon\}$ in $\text{FIRST}(X)$
 - (b) $\forall i : 1 < i \leq k$, if $\epsilon \in \text{FIRST}(Y_1) \cap \cdots \cap \text{FIRST}(Y_{i-1})$
(i.e., $Y_1 \cdots Y_{i-1} \Rightarrow^* \epsilon$)
then put $\text{FIRST}(Y_i) - \{\epsilon\}$ in $\text{FIRST}(X)$
 - (c) If $\epsilon \in \text{FIRST}(Y_1) \cap \cdots \cap \text{FIRST}(Y_k)$ then put ϵ in $\text{FIRST}(X)$
- Repeat until no more additions can be made.

FOLLOW

비단말 **A**에 대해, FOLLOW(**A**)

- 어떤 문장 형태 안에서 **A**의 바로 오른쪽에 나타날 수 있는 단말기호들의 집합

따라서 비단말의 FOLLOW 집합은 그 비단말 뒤에 문법적으로 올 수 있는 토큰들

단말기호에 대해서는 FOLLOW 집합을 정의하지 않음

FOLLOW

\$: End of token
(입력의 끝)

비단말 A 에 대해, $\text{FOLLOW}(A)$ 를 구하는 알고리즘

1. Put \$ in $\text{FOLLOW}(\langle \text{goal} \rangle)$
2. If $A \rightarrow \alpha B \beta$:
 - (a) Put $\text{FIRST}(\beta) - \{\epsilon\}$ in $\text{FOLLOW}(B)$
 - (b) If $\beta = \epsilon$ (i.e., $A \rightarrow \alpha B$) or $\epsilon \in \text{FIRST}(\beta)$ (i.e., $\beta \Rightarrow^* \epsilon$) then put $\text{FOLLOW}(A)$ in $\text{FOLLOW}(B)$

Repeat until no more additions can be made

LL(1) 문법

[기존 정의] 문법 G 는 LL(1):

- 모든 비단말기호 A 의

서로 다른 두 생산 규칙 $A \rightarrow \beta, A \rightarrow \gamma$ 에 대해

$$\text{FIRST}(\beta) \cap \text{FIRST}(\gamma) = \emptyset$$

예외 사례: $A \Rightarrow^* \epsilon$ 이면?

- 예: $A \rightarrow a \mid \epsilon$ 이면, $\{a\} \cap \{\epsilon\} = \emptyset$ 이지만,

$\text{FOLLOW}(A)$ 에 a 가 포함되면 두 생성 규칙 중 어느 것을 선택할지 결정할 수
없음

LL(1) 문법

[수정된 정의] 문법 G 는 LL(1):

- 모든 비단말기호 A 의

서로 다른 두 생산 규칙 $A \rightarrow \alpha_1, \dots, A \rightarrow \alpha_n$ 에 대해

1. $\text{FIRST}(\alpha_1), \dots, \text{FIRST}(\alpha_n)$ 의 어느 쌍을 선택해도 겹치지 않고(pairwise disjoint)

2. $\alpha_i \Rightarrow^* \epsilon$ 이면 $\text{FIRST}(\alpha_j) \cap \text{FOLLOW}(A) = \emptyset$, 모든 $j, 1 \leq j \leq n, i \neq j$

만일 G 의 언어에 ϵ 가 속하지 않으면(ϵ -free) 조건 1로 충분히 LL(1)을 정의할 수 있음

LL(1) 문법

LL(1) 문법에 대해 증명된 사실 (1/2)

- 예시:

$S \rightarrow a S \mid a$ LL(1)이 아님. $\text{FIRST}(a S) \cap \text{FIRST}(a) = \{ a \}$

$S \rightarrow a S'$ LL(1)이 맞음. 위 문법의 언어와 동일한 언어이지만.

$S' \rightarrow a S' \mid \epsilon$ $\text{Follow}(S') = \text{Follow}(S) = \{ \$ \} \neq \{ a \} = \text{First}(a S')$

LL(1) 파싱 테이블 만들기

입력: 문법 G

출력: 파싱 테이블 M

1. 모든 생산 규칙 $A \rightarrow \alpha$

1.1) 모든 $a \in \text{FIRST}(\alpha)$, $M[A, a]$ 에 $A \rightarrow \alpha$ 추가

1.2) 만약 $\epsilon \in \text{FIRST}(\alpha)$ 이면,

1.2.1) 모든 $b \in \text{FOLLOW}(A)$, $M[A, b]$ 에 $A \rightarrow \alpha$ 추가

1.2.2) 만약 $\$ \in \text{FOLLOW}(A)$ 이면, $M[A, \$]$ 에 $A \rightarrow \alpha$ 추가

2. M 의 나머지 채워지지 않은 곳, **error**

어떤 곳 $M[A, a]$ 가 둘 이상 채워지면,
입력 문법은 LL(1)이 아님

LL(1) 파싱 테이블 만들기 (예제)

문법 G3

$$\begin{array}{l|l} S \rightarrow E & T \rightarrow FT' \\ E \rightarrow TE' & T' \rightarrow *T \mid /T \mid \epsilon \\ E' \rightarrow +E \mid -E \mid \epsilon & F \rightarrow \text{id} \mid \text{num} \end{array}$$

	FIRST	FOLLOW
S	{num, id}	{}
E	{num, id}	{}
E'	{ε, +, -}	{}
T	{num, id}	{+, -, \$}
T'	{ε, *, /}	{+, -, \$}
F	{num, id}	{+, -, *, /, \$}
id	{id}	-
num	{num}	-
*	{*}	-
/	{/}	-
+	{+}	-
-	{-}	-

	id	num	+	-	*	/	\$
S	$S \rightarrow E$	$S \rightarrow E$	-	-	-	-	-
E	$E \rightarrow TE'$	$E \rightarrow TE'$	-	-	-	-	-
E'	-	-	$E' \rightarrow +E$	$E' \rightarrow -E$	-	-	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	$T \rightarrow FT'$	-	-	-	-	-
T'	-	-	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow *T$	$T' \rightarrow /T$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$	$F \rightarrow \text{num}$	-	-	-	-	-

LL(1)이 아닌 문법 예제

$\langle \text{stmt} \rangle ::= \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{stmt} \rangle \mid \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle \mid \dots$

좌인수 분해하면,

$\langle \text{stmt} \rangle ::= \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{stmt} \rangle \langle \text{stmt}' \rangle \mid \dots$

$\langle \text{stmt}' \rangle ::= \text{else } \langle \text{stmt} \rangle \mid \epsilon$

$\text{FIRST}(\langle \text{stmt}' \rangle) = \{ \epsilon, \text{else} \}$

$\text{FOLLOW}(\langle \text{stmt}' \rangle) = \{ \text{else}, \$ \}$

그런데, $\text{FIRST}(\langle \text{stmt}' \rangle) \cap \text{FOLLOW}(\langle \text{stmt}' \rangle) = \{ \text{else} \} \neq \emptyset$, 따라서 LL(1)이
아님!

Fix: $\langle \text{stmt}' \rangle ::= \text{else } \langle \text{stmt} \rangle$ 를 우선 선택하도록 하여 **else**를 가장 가까운 이전

에러 복구

핵심 개념

- 각 비단말 기호에 대해 파서가 동기화 할 수 있는 단말들의 집합을 구성
- **A**를 찾는 과정에서 오류가 발생하면, **SYNCH(A)**의 원소가 나올 때까지 입력을 건너뛰

SYNCH 구성 방법

- **FOLLOW(A)**의 각 단말 **a**를 **SYNCH(A)**에 추가 (예: 문장 끝 ;가 나올 때까지)
- 문장을 시작하는 키워드들도 **SYNCH(A)**에 추가 (예: 새 문장이 시작되는 곳까지)
- **FIRST(A)**의 기호들도 **SYNCH(A)**에 추가 (예: **A**의 생산 규칙에서 직접 나오는 곳까지)

에러 복구

에러 복구 방법: 스택 맨 위의 단말과 입력이 일치하지 않으면,

- 그 단말을 스택에서 **pop**함
- 그 단말이 삽입되었다고 가정했다는 메시지를 출력
- 그 다음 파싱을 계속

즉, $\text{SYNCH}(a) = V_t - \{ a \}$

목차

LL 파싱

[실습] MiniJava 파서 (JavaCC)

LR 파싱

The Java Compiler Compiler (JavaCC)

- Java용 Lex와 Yacc
- LR/LALR(1) 대신 LL(k) 방식 사용
- EBNF(Extended Backus-Naur Form)로 문법을 작성하고, LL(k) 파서를 생성
- 문법 안에 **Java** 액션 코드를 추가 작성 가능 (Yacc에서 C 액션 코드 작성 방식과 비슷)
- 필요하면 lookahead 길이 조정 가능
- 문법 명세 전체를 하나의 파일(.jj)에 작성

JavaCC 문법 명세 형식

문법 명세 전체를 하나의 파일(.jj)에 작성

- 헤더
- 어휘 분석을 위한 토큰 명세
- 문법 (+ 액션 코드)

```
[토큰] INTEGER_LITERAL: [1-9] [0-9]* | 0
```

```
TOKEN :
```

```
{
```

```
< INTEGER_LITERAL: ( ["1"-"9"] (["0"-"9"])* | "0" ) >
```

```
}
```

```
[문법] StatementListReturn -> Statement* return Expression ;
```

```
void StatementListReturn() :
```

```
{}
```

```
{
```

```
( Statement() )* "return" Expression() ";"
```

```
}
```

JavaCC 문법 명세 형식

교재 3.4 Using Parser Generators 참고 (JavaCC 버전 Grammar 3.31)

- void Assignment() : {} { Identifier() "=" Expression() ";" }

=> Assignment -> Identifier = Expression ;

- 왼쪽 기호 Assignment()

- 오른쪽 기호들 Identifier(), "=", Expression(), ";"

JavaCC 문법 명세 형식

교재 3.4 Using Parser Generators 참고 (문법 Grammar 3.30)

1 $P \rightarrow L$

2 $S \rightarrow \text{id} := \text{id}$

3 $S \rightarrow \text{while id do } S$

4 $S \rightarrow \text{begin } L \text{ end}$

5 $S \rightarrow \text{if id then } S$

6 $S \rightarrow \text{if id then } S \text{ else } S$

7 $L \rightarrow S$

8 $L \rightarrow L ; S$

JavaCC 문법 명세 형식

교재 3.4 Using Parser Generators 참고 (JavaCC 버전 Grammar 3.31)

```
PARSER_BEGIN(MyParser)
    public class MyParser {}
PARSER_END(MyParser)

SKIP :
{ " " | "\t" | "\n" }

TOKEN :
{ < WHILE: "while" >
| < BEGIN: "begin" >
| < END: "end" >
| < DO: "do" >
| < IF: "if" >
| < THEN: "then" >
| < ELSE: "else" >
| < SEMI: ";" >
| < ASSIGN: "=" >
| < ID: ["a"-"z"] (["a"-"z"] | ["0"-"9"])* >
}
```

```
void Prog() :
{
{ StmList() <EOF> }

void StmList() :
{
{ Stm() StmListPrime() }

void StmListPrime() :
{
{ ( ";" Stm() StmListPrime() )? }

void Stm() :
{
{ <ID> "=" <ID>
| "while" <ID> "do" Stm()
| "begin" StmList() "end"
| LOOKAHEAD(5) /* we need to lookahead till we see "else" */
"if" <ID> "then" Stm()
| "if" <ID> "then" Stm() "else" Stm()
}
```

JavaCC 문법 명세 형식

교재 3.4 Using Parser Generators 참고 (JavaCC 버전 Grammar 3.31)

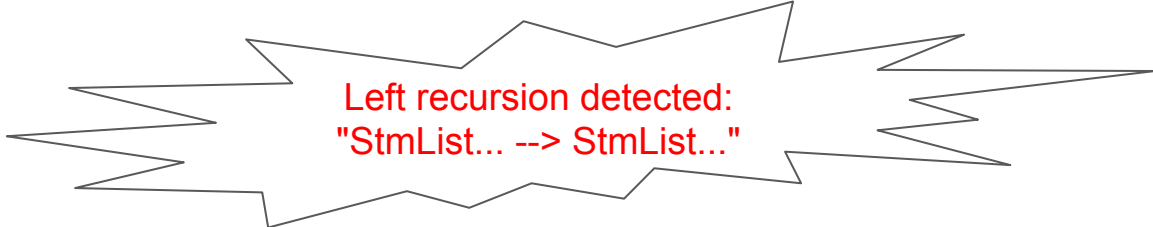
- void StmList () :

{ }

{ Stm ()

| **StmList()** “,” Stm()

}



Left recursion detected:
"StmList... --> StmList..."

JavaCC 문법 명세 형식

\$ javacc [MiniJavaLexer.jj](#) // 파서 코드를 생성

\$ javac *.java // 생성된 파서 코드를 컴파일

\$ java Main < Factorial.java // 컴파일된 파서를 실행

MiniJava LL(k) EBNF 문법

문법: chap3/MiniJava_EBNF.txt

Goal

`::= MainClass TypeDeclaration* EOF`

MainClass

```
::= "class" Identifier "{"  
    "public" "static" "void" "main"  
    "(" "String" "[" "]" Identifier ")"  
    "{"  
        ( LOOKAHEAD(2) VarDeclaration ) *  
        Statement *  
    }"  
}"
```

TypeDeclaration

```
::= LOOKAHEAD(3) ClassDeclaration  
| ClassExtendsDeclaration
```

- EBNF (Extended Backus-Naur Form)

```
MainClass -> class Identifier {  
    public static void main ( String [ ] Identifier )  
  
    { VarDeclaration* Statement* }  
}
```


MiniJava LL(k) EBNF 문법

문법: chap3/MiniJava_EBNF.txt

Goal

`::= MainClass TypeDeclaration* EOF`

MainClass

```
::= "class" Identifier "{"  
    "public" "static" "void" "main"  
    "(" "String" "[" "]" Identifier ")"  
    "{"  
        ( LOOKAHEAD(2) VarDeclaration )*  
        Statement*  
    "}"  
    "}"
```

TypeDeclaration

```
::= LOOKAHEAD(3) ClassDeclaration  
    | ClassExtendsDeclaration
```

- k-lookahead

TypeDeclaration

```
::= LOOKAHEAD(3) ClassDeclaration  
    | ClassExtendsDeclaration
```

- 예) `class C { ... }`

`class C extends D { ... }`

처음 두 토큰은 `class Identifier`까지 같으므로

그 다음 세 번째 토큰을 봐야 구문 가능:

- { 이면 `ClassDeclaration`,

`extends`이면 `ClassExtendsDeclaration`

MiniJava LL(k) EBNF 문법

- LOOKAHEAD(2) VarDeclaration() : 변수 선언인지, 일반 문장(assign)인지
- LOOKAHEAD(3) TypeDeclaration() : class C { 와 class C extends D { 구분
- LOOKAHEAD(3) Type() : int []와 int 구분
- LOOKAHEAD(2) AssignmentStatement / ArrayAssignmentStatement :
x = ... 와 x [...] = ... 구분
- LOOKAHEAD(“. “ Identifier() “(“) : .length와 .method() 구분
- LOOKAHEAD(3) ArrayAllocationExpression() : new int [...]와 new C () 구분

MiniJava LL(k) EBNF 문법

연산자 우선 순위를 고려한 문법

Expression
::= AndExpression

AndExpression
::= CompareExpression { "&&" CompareExpression }

CompareExpression
::= AddExpression { "<" AddExpression }

AddExpression
::= MulExpression { ("+" | "-") MulExpression }

MulExpression
::= UnaryExpression { "*" UnaryExpression }

UnaryExpression
::= "!" UnaryExpression
| PostfixExpression

PostfixExpression
::= PrimaryExpression
{
 "[" Expression "]"
 | LOOKAHEAD(2) "." "length"
 | LOOKAHEAD("." Identifier() "(") "." Identifier "(" [ExpressionList] ")"
}

ExpressionList
::= Expression { ExpressionRest }

ExpressionRest
::= "," Expression

PrimaryExpression
::= IntegerLiteral
| TrueLiteral
| FalseLiteral
| Identifier
| ThisExpression
| LOOKAHEAD(3) ArrayAllocationExpression
| AllocationExpression
| "(" Expression ")"

[실습] MiniJava 파서 만들기

- chap3에서 2장에서 작성한 `javacc` 폴더를 복사해와서 그 안의 [MiniJavaLexer.jj](#)에 MiniJava 문법을 추가 수정 작성해서 MiniJava 파서 만들기
- 목표: `programs/*.java` 8개 테스트 프로그램을 모두 성공적으로 파싱

목차

LL 파싱

[실습] MiniJava 파서 (JavaCC)

LR 파싱

용어 복습

$S \Rightarrow^* \alpha$ 이면, α 는 문장 형태(sentential form)

- $\alpha \in V_t$ 이면, α 는 $L(G)$ 의 문장(sentence)

- 아니면, (α 는 $L(G)$ 에 속하는 문장이 아닌) 문장 형태

왼쪽 문장 형태(Left-sentential form):

- 왼쪽 비단말기호 유도(leftmost derivation)로 나타나는 형태

오른쪽 문장 형태(Right-sentential form):

- 오른쪽 비단말기호 유도(rightmost derivation)로 나타나는 형태

상향 파싱(Bottom-up parsing)

목표

- 입력 문자열 w 와 문법 G 가 주어졌을때,
- 파스 트리를 리프(leaf)에서 시작하여 루트(root) 방향으로 만들기

파서는 언어의 우측 문장 형태(right-sentential form)를 현재까지 만들어진 트리의 상단 경계(upper frontier)와 반복적으로 매칭

각 매칭이 이루어질 때마다, 파서는 축약(reduction)을 적용하여 현재 경계를 확장

- 각 축약에서, 부분적으로 만들어진 트리들의 상단 경계와 생성 규칙의 RHS를 매치

- 그 경계 위에 새로운 노드(매치된 생성 규칙의 LHS) 하나를 추가

상향 파싱(Bottom-up parsing): 예제

문법

1: $S \rightarrow a A B e$

2: $A \rightarrow A b c$

3: $\quad \mid b$

4: $B \rightarrow d$

입력

a b b c d e

생산 (productions)	문장 형태 (sentential forms)
3: $A \rightarrow b$	a b b c d e
2: $A \rightarrow A b c$	a A b c d e
4: $B \rightarrow d$	a A d e
1: $S \rightarrow a A B e$	a A B e
-	S

핵심은 입력을 스캔하면서 올바른
문장 형태(sentential form)를

상향 파싱(Bottom-up parsing):

ϵ	a b b c d e \$	shift
a	b b c d e \$	shift
a b	b c d e \$	reduce
a A	b c d e \$	shift
a A b	c d e \$	shift
a A b c	d e \$	reduce
a A	d e \$	shift
a A d	e \$	reduce
a A B	e \$	shift
a A B e	e \$	reduce
S	\$	

Step 1

a

Step 2

a b

Step 3

a A
|
b

Step 4

a A b
|
b

Step 5

a A b c
|
b

Step 6

a A
/|\
A b c
|
b

Step 7

a A d
/|\
A b c
|
b

Step 8

a A B
/|\ |
A b c d
|
b

Step 9

a A B e
/|\ |
A b c d
|
b

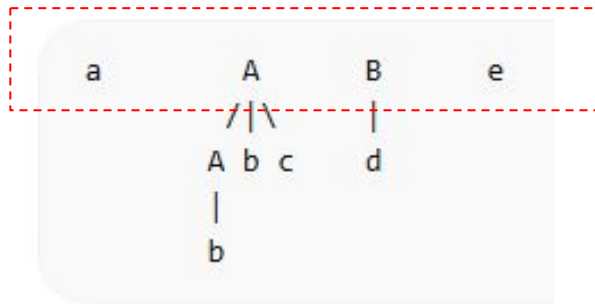
Step 10

S
/ | | \
a A B e
/|\ |
A b c d
|
b

상향 파싱(Bottom-up parsing):

상단 경계
(upper frontier)

Step 9



손잡이/핸들(Handles)

손잡이/핸들(Handles): 문자열 전체를 바로 줄이는 것이 아니라, 현재 문장 형태에서 지금 잡아서 축약할 수 있는 부분

찾고자 하는 것은 상단 경계(upper frontier)의 부분 문자열 α

- 이 부분 문자열과 일치하는 RHS를 갖는 생산 $A \rightarrow \alpha$
- α 를 A 로 축약할 수 있고, 이 과정은 오른쪽 비단말 유도의 역과정의 한 단계
- 이런 부분 문자열 α 를 손잡이/핸들이라고 함

손잡이/핸들(Handles)

손잡이/핸들(Handles): 문자열 전체를 바로 줄이는 것이 아니라, 현재 문장 형태에서 지금 잡아서 축약할 수 있는 부분

오른쪽 문장 형태 γ 의 핸들

- γ 의 핸들 : $A \rightarrow \beta$ 와 γ 안의 β 의 위치
- β 를 A 로 바꾸면 이전 오른쪽 문장 형태를 얻을 수 있어야 함

$$- S \Rightarrow_{rm}^* \alpha \quad A \quad w \Rightarrow_{rm} \alpha \quad \beta \quad w$$

$\alpha, \beta, \gamma \in V^*$ 단말/비단말 기호 열

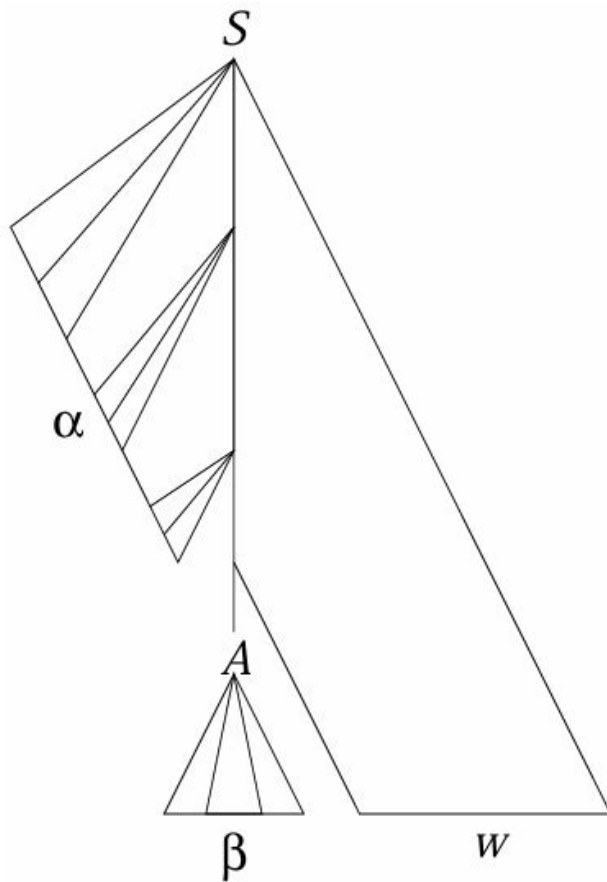
$u, v, w \in V_t^*$ 단말 기호 열

$$\text{예) } S \Rightarrow_{rm}^* aA \quad B \quad e \Rightarrow_{rm} aA \quad d \quad e$$

$$B \rightarrow d$$

손잡이/핸들(Handles)

α β w 의 핸들 $A \rightarrow \beta$



손잡이/핸들(Handles)

정리(Theorem)

- 문법 G 가 모호하지 않다면(unambiguous), 모든 우측 문장 형태(right-sentential form)는 유일한 손잡이/핸들(handle)을 갖음

증명

- 모호하지 않는 문법으로 우측 비단만 유도는 유일함
- γ_{i-1} 에서 γ_i 로 가기 위해 적용된 생성 규칙 $A \rightarrow \beta$ 도 유일
- 또한 $A \rightarrow \beta$ 가 적용되는 위치 k 도 유일
- 따라서, 손잡이/핸들 $A \rightarrow \beta$ 도 유일

손잡이/핸들(Handles)

증명 아이디어:

비모호한 문법에서는 한 문자열에 대해 파스 트리가 하나만 존재한다.

따라서 그 문자열을 만들어내는 우측 최다 유도(rightmost derivation) 도 하나만 존재한다.

그런데 **handle**은 바로 그 우측 최다 유도를 거꾸로 되돌릴 때 현재 단계에서 **reduce**해야 하는 부분이다.

우측 최다 유도가 유일하다면, 지금 단계에서 되돌려야 할 생성 규칙도 하나뿐이고, 그 규칙이 적용되는 위치도 하나뿐이다.

따라서 **handle**도 유일하다.

손잡이/핸들(Handles): 상향 파싱은 최우측 유도 역 과정

좌측 재귀 구조 문법 G2의 최우측 유도와 손잡이/핸들

1	$\langle \text{goal} \rangle$	$::=$	$\langle \text{expr} \rangle$
2	$\langle \text{expr} \rangle$	$::=$	$\langle \text{expr} \rangle + \langle \text{term} \rangle$
3		$ $	$\langle \text{expr} \rangle - \langle \text{term} \rangle$
4		$ $	$\langle \text{term} \rangle$
5	$\langle \text{term} \rangle$	$::=$	$\langle \text{term} \rangle * \langle \text{factor} \rangle$
6		$ $	$\langle \text{term} \rangle / \langle \text{factor} \rangle$
7		$ $	$\langle \text{factor} \rangle$
8	$\langle \text{factor} \rangle$	$::=$	num
9		$ $	id

Prod'n.	Sentential Form
–	$\langle \text{goal} \rangle$
1	$\langle \text{expr} \rangle$
3	$\langle \text{expr} \rangle - \langle \text{term} \rangle$
5	$\langle \text{expr} \rangle - \langle \text{term} \rangle * \langle \text{factor} \rangle$
9	$\langle \text{expr} \rangle - \langle \text{term} \rangle * \underline{\text{id}}$
7	$\langle \text{expr} \rangle - \underline{\langle \text{factor} \rangle} * \text{id}$
8	$\langle \text{expr} \rangle - \underline{\text{num}} * \text{id}$
4	$\underline{\langle \text{term} \rangle} - \text{num} * \text{id}$
7	$\underline{\langle \text{factor} \rangle} - \text{num} * \text{id}$
9	$\underline{\text{id}} - \text{num} * \text{id}$

손잡이/핸들 제거(Handle pruning)

손잡이/핸들 가지치기: 상향 파스 트리를 만들어가는 과정

최우측 유도(rightmost derivation)가 있다고 하면,

$$- S \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \dots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n \Rightarrow w$$

이 유도에서 상향 파스 트리를 구성하려면 i 를 n 으로 두고 다음 과정을 반복

- for $i = n$ downto 1

1. γ_i 안에서 핸들 $A \rightarrow \beta_i$ 를 찾음
2. γ_i 안에서 β_i 를 A 로 대체하여(축약) γ_{i-1} 로 변환

전체 과정 $2n$ 단계: 핸들 찾기 단계 + 축약 단계 (유도 길이 n)

스택 구현

손잡이/핸들 제거(Handle pruning)의
상향 파서를 구현하는 방법

- Shift-Reduce 파서

두 가지 요소

- 스택

- 입력 버퍼

절차

1. `stack <- $`

2. 스택 `top`이 `S` 시작 기호이고 입력
토큰이 `$`일 때까지 반복

- 2.1 손잡이/핸들이 없으면, `shift`

- 2.2 손잡이/핸들 `A -> β`를 `reduce`

- 2.2.1 스택에서 `β`를 `pop`

- 2.2.2 스택에 `A`를 `push`

예제: $x - 2 * y$

5 shift + 9 reduce + 1 accept

```

1 <goal> ::= <expr>
2 <expr> ::= <expr> + <term>
3           | <expr> - <term>
4           | <term>
5 <term>  ::= <term> * <factor>
6           | <term> / <factor>
7           | <factor>
8 <factor> ::= num
9           | id
    
```

Stack	Input	Action
\$	id - num * id	shift
\$id	- num * id	reduce 9
\$<factor>	- num * id	reduce 7
\$<term>	- num * id	reduce 4
\$<expr>	- num * id	shift
\$<expr> -	num * id	shift
\$<expr> - num	* id	reduce 8
\$<expr> - <factor>	* id	reduce 7
\$<expr> - <term>	* id	shift
\$<expr> - <term> *	id	shift
\$<expr> - <term> * id		reduce 9
\$<expr> - <term> * <factor>		reduce 5
\$<expr> - <term>		reduce 3
\$<expr>		reduce 1
\$<goal>		accept

1. 스택 Top이 손잡이/핸들의 오른쪽 끝일 때까지 Shift

2. 손잡이/핸들의 왼쪽 끝을 찾아 Reduce

Shift-reduce 파싱

Shift-reduce 파서는 이해하기 쉬움

Shift-reduce 파서의 4가지 액션

1. **shift**: 다음 입력 기호를 스택의 맨 위로 옮김
2. **reduce**: 스택 맨위의 **handle** 오른쪽 끝부터 **handle**의 왼쪽 끝을 찾아
handle을 스택에서 **pop** 하고,
해당 생성 규칙의 좌변(**LHS**)에 해당하는 비단말을 **push**
3. **accept**: 파싱을 종료하고 성공을 알린다.
4. **error**: 오류 복구 루틴(**error recovery routine**)을 호출한다.

핵심 문제: 손잡이/핸들을 어떻게 찾아내는가? (이 강의에서는 다루지 않음)

LR(k) 문법

[정의] 문법 G 가 LR(k):

- 최우측 유도(rightmost derivation)

$$S \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \dots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n \Rightarrow w$$

유도 과정의 각 오른쪽 문장 형태 γ_i 에 대해

1. 그 γ_i 의 손잡이/핸들을 찾아낼 수 있고,
2. 어떤 생성 규칙으로 축약할지 결정할 수 있으며
3. γ_i 를 왼쪽에서 오른쪽으로 진행하며 손잡이/핸들의 오른쪽 끝을 지나 최대 k 개 기호(k 개 lookahead)까지만 살펴보고 결정할 수 있어야 함

LR(k)

- L: Left-to-right
- R: Rightmost derivation in reverse
- k: 최대 k 개 lookahead

현재 상태에서 어디가 손잡이/핸들인지,
그리고 무슨 규칙으로 줄여야 하는지를
많지 않은 lookahead만으로 결정할 수
있어야 함.

LR(k) 문법

[정의] 문법 G 가 $L(k)$:

1. $S \Rightarrow_{rm}^* \alpha A w \Rightarrow_{rm} \alpha \beta w$ 이고
2. $S \Rightarrow_{rm}^* \gamma B x \Rightarrow_{rm} \alpha \beta y$ 이며
3. $FIRST_k(w) = FIRST_k(y)$ 일때
 $\Rightarrow \alpha A y = \gamma B x$

문장 형태 $\alpha \beta w$ 와 $\alpha \beta y$ 가 주어지고,

공통 접두사 $\alpha \beta$ 와 k-lookahead
 $FIRST_k(w) = FIRST_k(y)$ 를 가정,

$\alpha \beta w$ 는 $\alpha A w$ 로, $\alpha \beta y$ 는 $\gamma B y$ 로 축약

그러나, 공통 접두사를 가졌기 때문에
 $\alpha \beta y$ 는 $\alpha A y$ 로 축약되어,

$\alpha A y = \gamma B x$ 이다.

왜 LR 문법을 공부?

LR(1) 문법은 파서를 만들기 위해서 자주 사용됨. LR(1) 파서

- 가장 널리 사용되는 파서
- 거의 모든 자유 문맥 문법의 PL 구문을 표현할 수 있음
- LR 문법은 결정적(deterministic)·하향 파서가 처리할 수 있는 가장 일반적인 문법
- LR(1) 문법에 대해서는 고성능 파서를 손쉽게 구현할 수 있음
- 입력을 왼쪽에서 오른쪽으로 한 번 스캔하면서 가능한 가장 빨리 오류를 발견

왜 LR 문법을 공부?

LR(1) 문법은 파서를 만들기 위해서 자주 사용됨. LR(1) 파서

- LR 문법은 예측 파서(LL)가 인식할 수 있는 언어 집합을 모두 포함

LL(k) : $A \rightarrow \beta$ 의 β 의 k개 기호를 보고(lookahead) 무엇을 펼칠지를 결정

(무엇을 펼칠까?)

LR(k) : β 가 파생한 전체 문자열을 모두 읽은 다음, 그 뒤에 이어지는

k개 기호(look-ahead)를 함께 고려하여 β (핸들)을 축약 대상으로 인식

(무엇을 줄일까?)

왼편 vs. 오른편 재귀

오른쪽 재귀 구조

- predictive parser에서 종료(termination) 를 위해 필요
- 더 많은 스택 공간(stack space) 을 요구
- 우결합 연산자(right-associative operators)에 적합

왼쪽 재귀 구조

- 상향 파서에서 문제 없이 동작
- 필요한 스택 공간을 줄일 수 있음
- 좌결합 연산자(left-associative operators)에 적합

왼쪽 vs. 오른쪽 재귀

Rule of thumb

- 하향 파서에서는 오른쪽 재귀 구조의 문법
- 상향 파서에서는 왼쪽 재귀 구조의 문법

요약

재귀 하강(Recursive descent) 파서

- 문법(보통 LL(1) 문법)을 서로 재귀적으로 호출하는 일련의 프로시저를 직접 구현
- 따라서 recursive descent parser는 LL(1)의 언어적 제약 대부분을 그대로 갖음

LL(k) 파서: 어떤 생성 규칙의 사용을, 그 우변의 처음 k개 기호만 보고 인식

- $A \rightarrow \beta$ 와 $A \rightarrow \gamma$ 중 어느 것을 선택?

LR(k) 파서: 어떤 생성 규칙의 우변이 실제로 나타났음을, 그 우변으로부터 파생된 문자열 전체를 모두 읽은 뒤, 추가로 k개의 lookahead 기호를 보고 인식

- $A \rightarrow \beta$ 와 $B \rightarrow \beta$ 중 어느 것을 선택?